# x86 Architecture Overview

**Name: Matyas Abraham**
**IDNO: CTC-207-26**

# Introduction

This room is a crash course in x86 architecture, aimed specifically at the stuff that matters for reverse engineering and malware analysis. The idea is to build up a mental model of what's actually happening inside a CPU before jumping into disassemblers - registers, memory, and the stack all make a lot more sense once you've seen the bigger picture first.

x86 is the instruction set architecture behind most PCs, laptops, and servers today. It originally comes from Intel's 8086 processor back in the late 70s, and even though hardware has changed enormously since then, the naming and a lot of the underlying design has stuck around.

## CPU Architecture Overview

Most modern computers follow the Von Neumann architecture, which is basically the idea that a program's instructions (code) and the data it works with both live together in the same memory. The CPU pulls instructions out of that memory, runs them, and writes results back.

Inside the CPU itself, there are a handful of core building blocks:

- **Registers** - tiny, extremely fast storage slots built directly into the CPU. Much faster than RAM since there's no bus involved in reaching them.
- **ALU (Arithmetic Logic Unit)** - handles the actual math and logic: addition, subtraction, comparisons, AND/OR/XOR, and so on.
- **Control Unit** - directs the rest of the CPU, telling it how to move data around and carry out each instruction.
- **Instruction Pointer** - always points at the next instruction that's about to run.

Basically, everything a program does eventually comes down to moving data between memory and registers, and running some of it through the ALU.

***Challenge Questions***

**Q:** ***In the Von Neumann architecture, where are a program's code and data stored?***
**A:** Memory - they both live in the same memory space.

**Q:** ***Which part of the CPU stores small amounts of data?***
**A:** Registers

**Q:** ***Which unit performs arithmetic operations?***
**A:** The Arithmetic Logic Unit (ALU)

## Registers Overview

x86 has a set of general purpose registers, and depending on whether the system is running in 16-bit, 32-bit, or 64-bit mode, they go by slightly different names (the same physical register, just accessed at a different width):

| Register | Common Name | Typical Use |
|---|---|---|
| EAX / RAX | Accumulator | Arithmetic results |
| EBX / RBX | Base | Pointer to data |
| ECX / RCX | Counter | Loop counting |
| EDX / RDX | Data | Paired with EAX / I-O |

- **ESI / RSI, EDI / RDI** - Source and Destination Index. Used a lot for string and memory copy operations.
- **EBP / RBP** - the Base Pointer, marks the base of the current stack frame.
- **ESP / RSP** - the Stack Pointer, always points at the top of the stack.
- **EIP / RIP** - the Instruction Pointer, points at the next instruction to execute.

One of the bigger differences between 32-bit and 64-bit: 64-bit adds 8 extra general purpose registers, R8 through R15, that simply don't exist on a 32-bit system.

***Challenge Questions***

**Q:** ***Which register holds the address of the next instruction to be executed?***
**A:** The Instruction Pointer (IP / EIP / RIP)

**Q:** ***Which 32-bit register is also known as the Counter Register?***
**A:** ECX

**Q:** ***Which registers exist on a 64-bit system but not on a 32-bit one?***
**A:** R8-R15

## Registers Continued

On top of the general purpose registers, there's also the flags register (EFLAGS/RFLAGS) and a set of segment registers.

### Flags

- **Sign Flag (SF)** - set when the most significant bit of a result is 1, which generally means the result should be read as negative.
- **Zero Flag (ZF)** - set when the result of an operation is 0.
- **Trap Flag (TF)** - lets a debugger single-step through a program one instruction at a time. It's also how a program can detect it's being debugged, by checking whether this flag is set.

### Segment Registers

- **CS** - Code Segment. Points to where the executable instructions live in memory.
- **DS** - Data Segment.
- **SS** - Stack Segment.
- **ES, FS, GS** - extra segment registers. FS and GS specifically get used a lot on Windows to reach structures like the TEB and PEB.

***Challenge Questions***

**Q:** ***Which flag lets a program detect that it's being run inside a debugger?***
**A:** The Trap Flag (TF)

**Q:** ***Which flag gets set when the most significant bit of a result is 1?***
**A:** The Sign Flag (SF)

**Q:** ***Which segment register points to the code section in memory?***
**A:** CS (Code Segment)

## Memory Overview

A running program doesn't get to see the whole system's physical memory - it only sees its own virtual memory space, which the operating system maps behind the scenes. That's part of what keeps one process from being able to reach into another process's memory directly.

That virtual address space is broken into a few sections:

- **Code / Text segment** - the program's actual instructions.
- **Data segment** - global and static variables.
- **Heap** - memory allocated dynamically at runtime, and it can grow as the program asks for more.

- **Stack** - handles function calls, local variables, and return addresses. This is also the part that drives the program's control flow, since it's what tells the CPU where to jump back to once a function finishes.

***Challenge Questions***

**Q:** ***Does a running program have a full view of the system's physical memory? (Y/N)***
**A:** N - it only sees its own virtual memory space.

**Q:** ***Which section of memory contains the executable code?***
**A:** The Code (Text) segment

**Q:** ***Which memory section is tied to the program's control flow?***
**A:** The Stack

## Stack Layout

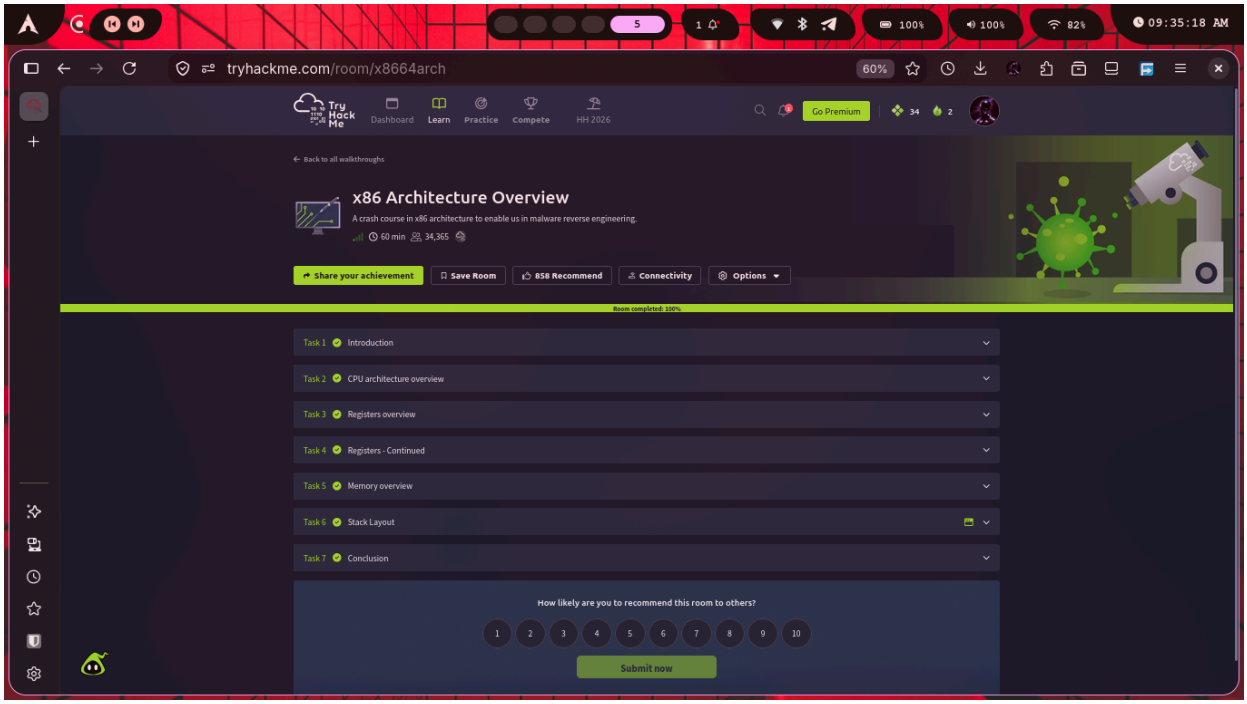
The stack is what actually makes function calls work. Every time a function gets called, a new stack frame is pushed on: the return address, the saved base pointer, and room for local variables. Once the function finishes, all of that gets popped back off and the CPU jumps back to whatever return address was saved.

This task has a small interactive exercise that walks through pushing values onto the stack until it overflows - which is the classic idea behind a stack buffer overflow. If a program writes more data into a local buffer than it actually has room for, that extra data starts spilling into whatever is sitting above it on the stack, including the saved return address. If an attacker can control what gets written there, they can potentially redirect execution to wherever they want once the function returns.

Completing the exercise on the static site gives a flag:

***Challenge Questions***

**Q:** ***Follow the instructions in the attached static site - what's the flag?***
**A:** THM{SMASHED_THE_STACK}


tryhackme.com/room/x8664arch
Try Hack Me
Dashboard
Learn
Practice
Compete
HH 2026
Go Premium
Back to all walkthroughs
x86 Architecture Overview
A crash course in x86 architecture to enable us in malware reverse engineering.
60 min
34,365
Share your achievement
Save Room
858 Recommend
Connectivity
Options
Room completed: 100%
Task 1 Introduction
Task 2 CPU architecture overview
Task 3 Registers overview
Task 4 Registers - Continued
Task 5 Memory overview
Task 6 Stack Layout
Task 7 Conclusion
How likely are you to recommend this room to others?
1
2
3
4
5
6
7
8
9
10
Submit now